

Deep signature transforms for rough volatility model

1 Supervised learning with fractional Brownian motion and rough Bergomi volatility process

1.1 Implementation details

In the following numerical experiments, we will introduce various neural network models to calibrate parameters from time series data generated by fractional Brownian motions and rough Bergomi volatility processes.

1.1.1 Models' architecture

We first clarify the architecture of these neural network models that we will use in the following numerical experiments.

The DeepSigNet model is a neural network model with signature transform proposed by Kidger, Bonnier, Perez Arribas, Salvi and Lyons (2019). The CNN model is a convolutional neural network proposed by Stone (2020). And we also provide another maxpooling CNN model and signature CNN model to explore whether the signature transform is a better choice than the maxpooling layer. The specific structures of these models are presented below:

- The DeepSigNet model:
 - a convolutional layer with 3 channels and kernel size 3;
 - augmented with time and original values;
 - the signature transform with truncation order $N = 3$;
 - a fully connected feedforward neural network with 5 hidden layers, each of size 32 and ReLU activation;
 - the non-linear transform with sigmoid function.
- The CNN model:
 - a convolutional layer with 32 channels and kernel size 20;
 - the leaky ReLU transform with negative slope 0.1;
 - a max pooling layer with size 3;

- a dropout layer with rate 0.25;
 - a convolutional layer with 64 channels and kernel size 20;
 - the leaky ReLu transform with negative slope 0.3;
 - a max pooling layer with size 3;
 - a dropout layer with rate 0.25;
 - a convolutional layer with 128 channels and kernel size 20;
 - the leaky ReLu transform with negative slope 0.1;
 - a max pooling layer with size 3;
 - a dropout layer with rate 0.4;
 - a dense layer with 128 units;
 - the leaky ReLu transform with negative slope 0.1;
 - a dropout layer with rate 0.3;
 - a dense layer with 1 units;
 - the non-linear transform with sigmoid function.
- The maxpooling CNN model:
 - a convolutional layer with 32 channels and kernel size 20;
 - the leaky ReLu transform with negative slope 0.1;
 - a max pooling layer with size 3;
 - a dropout layer with rate 0.25;
 - a dense layer with 128 units;
 - the leaky ReLu transform with negative slope 0.1;
 - a dropout layer with rate 0.25;
 - a dense layer with 1 units;
 - the non-linear transform with sigmoid function.
 - The signature CNN model:
 - a convolutional layer with 32 channels and kernel size 20;
 - the leaky ReLu transform with negative slope 0.1;
 - the signature transform with truncation order $N = 2$;
 - a dropout layer with rate 0.25;
 - a dense layer with 128 units;
 - the leaky ReLu transform with negative slope 0.1;
 - a dropout layer with rate 0.25;
 - a dense layer with 1 units;
 - the non-linear transform with sigmoid function.

1.1.2 Simulated data set

We use simulated sample paths of fractional Brownian motions (fBm) $\{Z_t\}$ and rough Bergomi (rBergomi) volatility processes $\{V_t\}$ as the input data. The fractional Brownian motion is defined as

$$Z_t = \int_0^t (t-s)^{H-\frac{1}{2}} dW_s, \quad t > 0, \quad (1)$$

where $H \in (0, 1)$ is the Hurst parameter and W is a standard Brownian motion. The rBergomi volatility process is defined as

$$V_t = V_0 \exp\left(\eta\sqrt{2H}Z_t - \frac{\eta^2}{2}t^{2H}\right), \quad V_0 > 0, \quad (2)$$

where η is a strictly positive constant.

To simulate these sample paths we use Cholesky decomposition; this very well-known simulation technique is recommended because the resulting sample paths have the exact distribution, rather than an approximate distribution, of the normalised log volatility process of the rBergomi model.

1.2 Numerical examples

In this example, we first let the Hurst parameter H takes values in the discrete grid $\{0.1, 0.2, 0.3, 0.4, 0.5\}$ and we also sample 5 H values from different probability distributions: the Uniform distribution¹ on $(0, 0.5)$ and the Beta(1, 9) distribution². Then we use these different H to simulate fBm and rBergomi paths ($\eta = 1$) with different number of time steps $\{100, 200, 300, 400, 500\}$ to generate five corresponding training/test data sets and the sizes of each data set are given in Table 1.

Table 1: Training and test data size.

Data set	Number of samples
Training set	2800
Test set	700

We use adaptive moment estimation (Adam) to train the networks with the architecture³ described in Section 1.1, setting *batch size* = 64, *epochs* = 100 and *learning rate* = 0.001 as is fairly standard. We use the mean square error (MSE) as the loss function for training and we also provide the root mean square

¹The corresponding set of possible H values is $\{0.05, 0.18, 0.29, 0.31, 0.44\}$.

²The corresponding set of possible H values is $\{0.02, 0.06, 0.07, 0.13, 0.22\}$.

³The code for DeepSigNet model is provided by the original paper and My DeepSigNet model has the same structure with DeepSigNet but recoded by myself with Pytorch. The CNN is also recoded by Pytorch with the same structure as the original paper.

error (RMSE) in test results as reference. The test results are shown in Table 2-4 and the loss plots are shown in Appendix A.

There are some remarks for the results below:

- The number of parameters in DeepSigNet remains stable regardless of the input length, indicating that the model's complexity remains stable across different input lengths.
- My DeepSigNet slightly outperforms DeepSigNet in most cases. But both of them outperform than CNN including the results in Stone (2020).
- The Maxpooling CNN slightly outperforms Signature CNN but significantly underperforms DeepSigNet. This may due to the fact that the first convolutional layer of Signature CNN significantly increases the number of sequence features used for signature transform.

Table 2: Test results for discretised H .

Models	Input length	fBm		rBergomi		#Params
		MSE	RMSE	MSE	RMSE	
DeepSigNet	100	1.18×10^{-4}	1.08×10^{-2}	8.87×10^{-4}	2.84×10^{-2}	9261
	200	3.00×10^{-5}	5.36×10^{-3}	4.51×10^{-4}	2.10×10^{-2}	9261
	300	3.63×10^{-5}	5.94×10^{-3}	2.43×10^{-4}	1.54×10^{-2}	9261
	400	4.48×10^{-5}	6.61×10^{-3}	1.85×10^{-4}	1.33×10^{-2}	9261
	500	3.75×10^{-5}	6.02×10^{-3}	2.14×10^{-4}	1.45×10^{-2}	9261
My DeepSigNet	100	1.09×10^{-5}	2.05×10^{-3}	4.96×10^{-4}	2.22×10^{-2}	9261
	200	2.59×10^{-6}	1.55×10^{-3}	2.48×10^{-4}	1.56×10^{-2}	9261
	300	1.38×10^{-5}	3.67×10^{-3}	2.33×10^{-4}	1.52×10^{-2}	9261
	400	2.55×10^{-6}	1.49×10^{-3}	1.95×10^{-4}	1.38×10^{-2}	9261
	500	6.88×10^{-6}	2.34×10^{-3}	1.18×10^{-4}	1.07×10^{-2}	9261
CNN	100	2.11×10^{-3}	4.59×10^{-2}	2.51×10^{-3}	4.97×10^{-2}	255073
	200	2.21×10^{-3}	4.69×10^{-2}	1.57×10^{-3}	3.96×10^{-2}	320609
	300	2.66×10^{-3}	5.16×10^{-2}	2.16×10^{-3}	4.62×10^{-2}	386145
	400	2.44×10^{-3}	4.93×10^{-2}	1.51×10^{-3}	3.88×10^{-2}	435297
	500	1.84×10^{-3}	4.29×10^{-2}	1.88×10^{-3}	4.32×10^{-2}	500833
Maxpooling CNN	100	1.90×10^{-3}	4.36×10^{-2}	1.70×10^{-3}	4.11×10^{-2}	136097
	200	2.00×10^{-3}	4.46×10^{-2}	1.28×10^{-3}	3.57×10^{-2}	271265
	300	2.49×10^{-3}	4.98×10^{-2}	1.81×10^{-3}	4.24×10^{-2}	410529
	400	2.01×10^{-3}	4.47×10^{-2}	1.27×10^{-3}	3.56×10^{-2}	545697
	500	1.29×10^{-3}	3.58×10^{-2}	1.38×10^{-3}	3.70×10^{-2}	680865
Signature CNN	100	2.51×10^{-3}	5.00×10^{-2}	3.40×10^{-3}	5.81×10^{-2}	136097
	200	2.46×10^{-3}	4.95×10^{-2}	3.95×10^{-3}	6.28×10^{-2}	136097
	300	2.07×10^{-3}	4.54×10^{-2}	2.96×10^{-3}	5.42×10^{-2}	136097
	400	3.26×10^{-3}	5.70×10^{-2}	2.97×10^{-3}	5.43×10^{-2}	136097
	500	2.56×10^{-3}	5.05×10^{-2}	2.96×10^{-3}	5.41×10^{-2}	136097

Table 3: Test results for $H \sim \text{Uniform}(0, 0.5)$.

Models	Input length	fBm		rBergomi		#Params
		MSE	RMSE	MSE	RMSE	
DeepSigNet	100	8.22×10^{-5}	9.00×10^{-3}	1.25×10^{-3}	3.52×10^{-2}	9261
	200	1.61×10^{-4}	1.25×10^{-2}	3.18×10^{-4}	1.75×10^{-2}	9261
	300	5.58×10^{-4}	2.35×10^{-2}	4.94×10^{-4}	2.21×10^{-2}	9261
	400	5.56×10^{-4}	2.35×10^{-2}	2.05×10^{-4}	1.42×10^{-2}	9261
	500	5.14×10^{-4}	2.25×10^{-2}	1.07×10^{-3}	2.23×10^{-2}	9261
My DeepSigNet	100	3.07×10^{-5}	5.48×10^{-3}	8.47×10^{-4}	2.89×10^{-2}	9261
	200	1.95×10^{-5}	4.39×10^{-3}	2.13×10^{-4}	1.44×10^{-2}	9261
	300	4.26×10^{-5}	6.40×10^{-3}	4.91×10^{-4}	2.19×10^{-2}	9261
	400	1.65×10^{-5}	4.04×10^{-3}	1.25×10^{-4}	1.11×10^{-2}	9261
	500	7.78×10^{-5}	8.48×10^{-3}	4.36×10^{-4}	1.59×10^{-2}	9261
CNN	100	1.90×10^{-3}	4.35×10^{-2}	2.83×10^{-3}	5.31×10^{-2}	255073
	200	1.81×10^{-3}	4.25×10^{-2}	1.66×10^{-3}	4.06×10^{-2}	320609
	300	1.62×10^{-3}	4.02×10^{-2}	2.15×10^{-3}	4.63×10^{-2}	386145
	400	1.23×10^{-2}	1.10×10^{-1}	2.45×10^{-3}	4.94×10^{-2}	435297
	500	1.96×10^{-3}	4.42×10^{-2}	1.64×10^{-3}	4.05×10^{-2}	500833
Maxpooling CNN	100	1.10×10^{-3}	3.31×10^{-2}	2.74×10^{-3}	5.23×10^{-2}	136097
	200	1.35×10^{-3}	3.67×10^{-2}	1.60×10^{-3}	3.99×10^{-2}	271265
	300	1.20×10^{-3}	3.46×10^{-2}	1.50×10^{-3}	3.86×10^{-2}	410529
	400	8.62×10^{-4}	2.93×10^{-2}	1.50×10^{-3}	3.85×10^{-2}	545697
	500	5.94×10^{-4}	2.42×10^{-2}	1.36×10^{-3}	3.67×10^{-2}	680865
Signature CNN	100	1.72×10^{-3}	4.14×10^{-2}	2.83×10^{-3}	5.30×10^{-2}	136097
	200	1.69×10^{-3}	4.10×10^{-2}	2.89×10^{-3}	5.36×10^{-2}	136097
	300	1.68×10^{-3}	4.10×10^{-2}	3.33×10^{-3}	5.76×10^{-2}	136097
	400	1.86×10^{-3}	4.30×10^{-2}	3.28×10^{-3}	5.71×10^{-2}	136097
	500	1.77×10^{-3}	4.20×10^{-2}	3.03×10^{-3}	5.49×10^{-2}	136097

Table 4: Test results for $H \sim \text{Beta}(1, 9)$.

Models	Input length	fBm		rBergomi		#Params
		MSE	RMSE	MSE	RMSE	
DeepSigNet	100	4.83×10^{-5}	6.89×10^{-3}	5.86×10^{-4}	2.40×10^{-2}	9261
	200	2.85×10^{-5}	5.29×10^{-3}	2.37×10^{-4}	1.53×10^{-2}	9261
	300	2.39×10^{-5}	4.88×10^{-3}	2.11×10^{-4}	1.44×10^{-2}	9261
	400	1.01×10^{-5}	3.15×10^{-3}	1.89×10^{-4}	1.37×10^{-2}	9261
	500	2.60×10^{-4}	1.61×10^{-2}	9.48×10^{-5}	9.66×10^{-3}	9261
My DeepSigNet	100	5.33×10^{-5}	7.20×10^{-3}	4.51×10^{-4}	2.11×10^{-2}	9261
	200	6.94×10^{-4}	2.58×10^{-2}	1.94×10^{-4}	1.38×10^{-2}	9261
	300	1.36×10^{-4}	1.15×10^{-2}	9.44×10^{-5}	9.52×10^{-3}	9261
	400	1.57×10^{-4}	1.21×10^{-2}	1.08×10^{-4}	1.03×10^{-2}	9261
	500	3.10×10^{-3}	5.39×10^{-2}	3.45×10^{-5}	5.78×10^{-3}	9261
CNN	100	1.13×10^{-3}	3.36×10^{-2}	1.47×10^{-2}	1.21×10^{-1}	255073
	200	1.32×10^{-3}	3.64×10^{-2}	1.38×10^{-2}	1.17×10^{-1}	320609
	300	1.22×10^{-3}	3.48×10^{-2}	1.56×10^{-2}	1.24×10^{-1}	386145
	400	1.22×10^{-3}	3.49×10^{-2}	1.52×10^{-2}	1.23×10^{-1}	435297
	500	1.10×10^{-3}	3.32×10^{-2}	1.61×10^{-2}	1.26×10^{-1}	500833
Maxpooling CNN	100	3.19×10^{-4}	1.78×10^{-2}	6.67×10^{-4}	2.58×10^{-2}	136097
	200	2.65×10^{-4}	1.62×10^{-2}	4.21×10^{-4}	2.05×10^{-2}	271265
	300	3.30×10^{-4}	1.81×10^{-2}	4.09×10^{-4}	2.01×10^{-2}	410529
	400	1.48×10^{-2}	1.21×10^{-1}	3.46×10^{-4}	1.86×10^{-2}	545697
	500	1.45×10^{-2}	1.20×10^{-1}	4.42×10^{-4}	2.10×10^{-2}	680865
Signature CNN	100	7.74×10^{-4}	2.78×10^{-2}	9.91×10^{-4}	3.14×10^{-2}	136097
	200	1.34×10^{-2}	1.15×10^{-1}	7.74×10^{-4}	2.78×10^{-2}	136097
	300	1.31×10^{-2}	1.15×10^{-1}	7.63×10^{-4}	2.75×10^{-2}	136097
	400	1.48×10^{-2}	1.21×10^{-1}	6.97×10^{-4}	2.63×10^{-2}	136097
	500	1.45×10^{-2}	1.20×10^{-1}	7.49×10^{-4}	2.74×10^{-2}	136097

2 Calibration with signature transformer models

2.1 Signature & Transformer

Given a path $X = \{x_t | x_t = (x_t^1, \dots, x_t^d) \in \mathbb{R}^d, t \in [a, b]\}$ with $a, b \in \mathbb{R}$ and $d \in \mathbb{N}^+$, we can define its signature by iterated integrals as follows

$$\text{Sig}(X) = \left(\int_{a < t_1 < \dots < t_k < b} dx_{t_1} \otimes \dots \otimes dx_{t_k} \right)_{k \geq 1}$$

$$= \left(\left(\int \cdots \int_{a < t_1 < \cdots < t_k < b} dx_{t_1}^{d_1} \otimes \cdots \otimes dx_{t_k}^{d_k} \right)_{1 \leq d_1, \dots, d_k \leq d} \right)_{k \geq 1}, \quad (3)$$

where $\otimes$ denotes the tensor product. From equation above, we can easily find that the i -th signature of path X is $\int \cdots \int_{a < t_1 < \cdots < t_i < b} dx_{t_1} \otimes \cdots \otimes dx_{t_i}$, $i = 1, 2, \dots$

and we can also define the corresponding truncated signature as

$$\text{Sig}^N(X) = \left(\int \cdots \int_{a < t_1 < \cdots < t_k < b} dx_{t_1} \otimes \cdots \otimes dx_{t_k} \right)_{1 \leq k \leq N}, \quad (4)$$

with the truncation order N .

Transformer is first proposed by Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser L. and Polosukhin, I. (2017). The key component in transformer is self-attention mechanism. To clarify this, we discretize the path X with $n \in \mathbb{N}^+$ time grids as $X^n = (x_{t_1}, \dots, x_{t_n}) \in \mathbb{R}^{n \times d}$, $a = t_1 < \cdots < t_n = b$. Let $W_Q \in \mathbb{R}^{d_{\text{attn}} \times d}$, $W_K \in \mathbb{R}^{d_{\text{attn}} \times d}$ and $W_V \in \mathbb{R}^{d_{\text{attn}} \times d}$ with $d_{\text{attn}} \in \mathbb{N}^+$, then we can define the query, key and value in self-attention mechanism as $Q = X^n W_Q^T$, $K = X^n W_K^T$ and $V = X^n W_V^T$, respectively, which are linear projections from the path. Then the self-attention can be defined as

$$\text{Self-Attention}(X^n; W_Q, W_K, W_V) := \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right) V. \quad (5)$$

Intuitively, self-attention can be viewed as the combination of linear and non-linear transformation applied to the input X^n which is similar to the multilayer perceptron (MLP).

If we compare signature transform with self-attention mechanism, there is only one specific $\text{Sig}(X^n)$ corresponding to the path X^n while there are countless $\text{Self-Attention}(X^n; W_Q, W_K, W_V)$ with different weight matrix W_Q , W_K and W_V . The transformer, including the self-attention mechanism, is a typical neural network that can be trained while signature transform is a feature map that transforms paths into fixed-dimensional vectors capturing important statistical information.

To combine signature transform with transformer, we need to preserve the stream-like nature of input data because signature transform will consume it. To illustrate this, we compare the dimension of input path $X^n \in \mathbb{R}^{n \times d}$ and output $\text{Sig}^N(X^n) \in \mathbb{R}^{\frac{d^{N+1}-1}{d-1}-1}$, there is no underlying path in $\text{Sig}^N(X^n)$ i.e. without stream-like nature. The solution is to lift the input data as

$$\begin{aligned} l(X^n) &= (l_2(X^n), l_3(X^n), \dots, l_n(X^n)) \\ &= ((x_{t_1}, x_{t_2}), (x_{t_1}, x_{t_2}, x_{t_3}), \dots, (x_{t_1}, \dots, x_{t_n})), \end{aligned} \quad (6)$$

where $l_i(X^n) := (x_{t_1}, \dots, x_{t_i})$, $i = 2, 3, \dots, n$. Then the output becomes to $\text{Sig}^N(l(X^n)) \in \mathbb{R}^{(n-1) \times (\frac{d^{N+1}-1}{d-1}-1)}$.

2.2 Models' architecture

In this section, we clarify the architecture of three signature transformer models that we will use in the following numerical experiments.

- The SigFormer_1 model:
 - a convolutional layer with 3 channels and kernel size 3;
 - augmented with time and original values;
 - lifted as equation (6);
 - the signature transform with truncation order $N = 3$;
 - the transformer layer with self-attention mechanism;
 - a fully connected feedforward neural network with 5 hidden layers, each of size 32 and ReLU activation;
 - the non-linear transform with sigmoid function.
- The SigFormer_2 model:
 - a convolutional layer with 3 channels and kernel size 3;
 - augmented with time and original values;
 - lifted as equation (6);
 - the signature transform with truncation order $N = 3$;
 - the transformer layer with self-attention mechanism;
 - a linear transform layer;
 - the non-linear transform with sigmoid function.
- The SigFormer_3 model:
 - augmented with time;
 - the transformer layer with self-attention mechanism;
 - lifted as equation (6);
 - the signature transform with truncation order $N = 3$;
 - a fully connected feedforward neural network with 5 hidden layers, each of size 32 and ReLU activation;
 - the non-linear transform with sigmoid function.

In addition to these three models, we also introduce My DeepSigNet and CNN model in Section 1.1 for comparison.

2.3 Numerical examples

In this example, we use the same setting as Section 1.1 but increase the training epochs to 150. The results are shown in Table 5 and Figure 2.3.

Table 5: Test results for $H \sim \text{Beta}(1, 9)$.

Models	Input length	rBergomi		#Params
		MSE	RMSE	
CNN	100	1.47×10^{-2}	1.21×10^{-1}	255073
My DeepSigNet	100	5.27×10^{-4}	2.29×10^{-2}	9261
SigFormer_1	100	4.97×10^{-4}	2.20×10^{-2}	567416
SigFormer_2	100	5.46×10^{-4}	2.33×10^{-2}	87433
SigFormer_3	100	4.85×10^{-3}	6.95×10^{-2}	495359

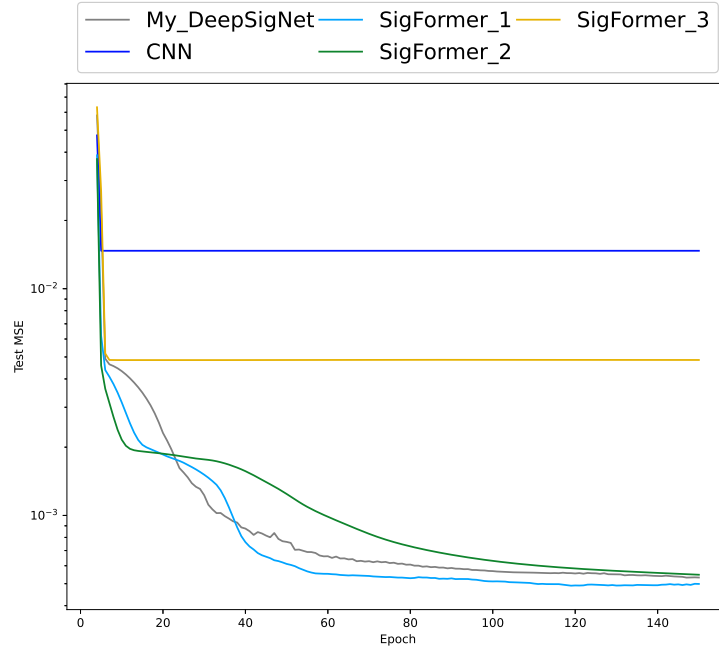


Figure 1: Loss plot for rBergomi volatility processes with $H \sim \text{Beta}(1, 9)$. The data has been taken the rolling mean with a window size of 5.

References

- Kidger, P., Bonnier, P., Perez Arribas, I., Salvi, C., and Lyons, T. (2019). Deep signature transforms. *Advances in Neural Information Processing Systems*, 32.
- Stone, H. (2020). Calibrating rough volatility models: a convolutional neural network approach. *Quantitative Finance*, 20(3), 379-392.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser L. and Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.

A Loss plot for fBm and rBergomi

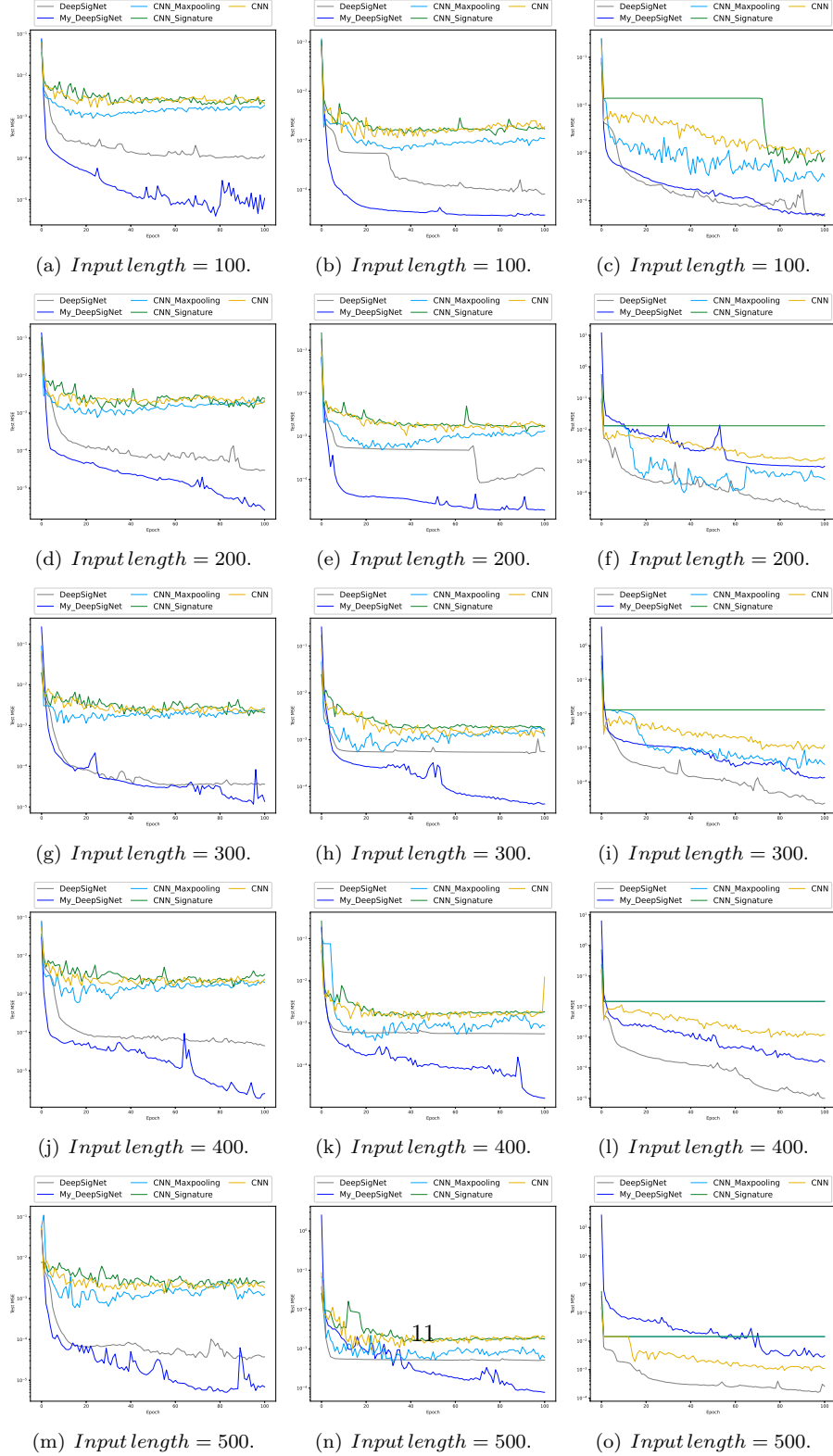


Figure 2: Loss plot for fractional Brownian motions with (Left:) discretised H ; (Middle:) $H \sim \text{Uniform}(0, 0.5)$; (Right:) $H \sim \text{Beta}(1, 9)$.

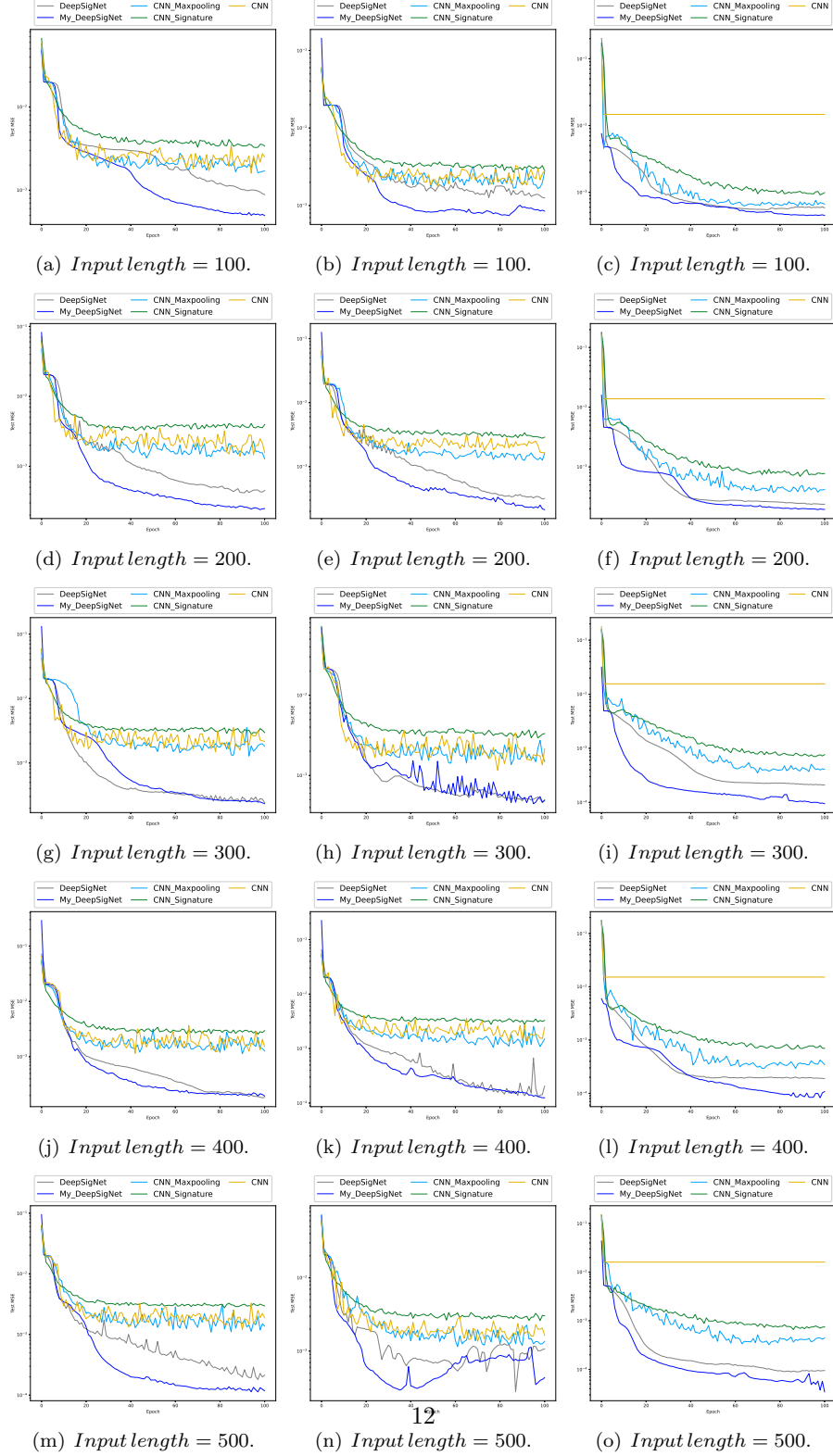


Figure 3: Loss plot for rBergomi volatility processes with (Left:) discretised H ; (Middle:) $H \sim \text{Uniform}(0, 0.5)$; (Right:) $H \sim \text{Beta}(1, 9)$.